Assgnment 6:

```
/**
 *
================================================================================
 * KBTCOE Connect — Student Directory Application Logic (app.js)
 * Assignment 06: JavaScript Basics
 *
 * Demonstrates:
 * 1. Variables: let vs const
 * 2. Data Types: String, Number, Boolean, Array, Object
 * 3. Operators: Arithmetic, Comparison (===), Logical (&&, ||, !)
 * 4. Conditionals: if / else if / else and ternary operator (? :)
 * 5. Functions: Regular functions, arrow functions, default parameters
 * 6. Array Methods: map(), filter(), find(), reduce()
 * 7. Object Manipulation: Property access, immutable state updates
 * 8. DOM Manipulation: createElement(), textContent, appendChild(), event
listeners
 * 9. Template Literals & dynamic rendering
 * 10. Error Handling: try...catch blocks
 *
================================================================================
 */

"use strict";

//
================================================================================
// 1. DATASET: Fictional KBTCOE Student Directory (Synthetic Data)
//
================================================================================
/**
 * Array of student objects demonstrating data types:
 * - Number: id, cgpa
 * - String: studentId, name, department, deptCode, year, bio, achievement
 * - Boolean: isFollowing
 * - Array: skills
 * - Object: Each student record is an object with structured key-value pairs
 */
const initialStudents = [
  {
    id: 1,
    studentId: "KBT-2023-CS01",
    name: "Aarav Sharma",
    department: "Computer Engineering",
    deptCode: "comp",
    year: "Final Year (BE)",
    cgpa: 9.15,
    skills: ["React", "Node.js", "TypeScript", "Docker"],
    bio: "Full-stack developer passionate about building scalable cloud
architectures and open-source developer tooling.",
```

```
    achievement: "Winner - Smart India Hackathon (SIH) 2025 Grand Finale",
    isFollowing: false
  },
  {
    id: 2,
    studentId: "KBT-2023-IT04",
    name: "Pooja Deshmukh",
    department: "Information Technology",
    deptCode: "it",
    year: "Third Year (TE)",
    cgpa: 8.84,
    skills: ["Python", "Machine Learning", "FastAPI", "PostgreSQL"],
    bio: "Aspiring data scientist exploring applied NLP and computer vision
pipelines for agricultural technology.",
    achievement: "Published IEEE Conference Paper on Crop Disease Detection",
    isFollowing: false
  },
  {
    id: 3,
    studentId: "KBT-2024-AI02",
    name: "Rohan Kulkarni",
    department: "Artificial Intelligence & Data Science",
    deptCode: "aids",
    year: "Third Year (TE)",
    cgpa: 9.32,
    skills: ["Deep Learning", "PyTorch", "Computer Vision", "Python"],
    bio: "Research-driven AI developer working on multimodal LLMs and autonomous
robotics vision systems.",
    achievement: "1st Runner-up at National AI Conclave Hackathon 2025",
    isFollowing: false
  },
  {
    id: 4,
    studentId: "KBT-2023-ET07",
    name: "Sneha Patil",
    department: "Electronics & Telecommunication",
    deptCode: "entc",
    year: "Final Year (BE)",
    cgpa: 8.65,
    skills: ["Embedded C", "IoT", "VLSI", "MATLAB", "Arduino"],
    bio: "Hardware-software integration enthusiast specializing in smart city
telemetry and industrial IoT sensor nodes.",
    achievement: "Best Technical Project Award at TechFest KBTCOE 2024",
    isFollowing: false
  },
  {
    id: 5,
    studentId: "KBT-2023-ME03",
    name: "Aditya Jadhav",
    department: "Mechanical Engineering",
    deptCode: "mech",
    year: "Final Year (BE)",
    cgpa: 8.42,
    skills: ["SolidWorks", "ANSYS", "AutoCAD", "Python", "3D Printing"],
```

```
    bio: "Mechanical design engineer focused on finite element analysis (FEA),
electric vehicle chassis design, and CFD simulation.",
    achievement: "Team Lead - SAE BAJA Electric Vehicle Competition 2025",
    isFollowing: false
  },
  {
    id: 6,
    studentId: "KBT-2024-CV05",
    name: "Neha Joshi",
    department: "Civil Engineering",
    deptCode: "civil",
    year: "Second Year (SE)",
    cgpa: 8.78,
    skills: ["STAAD Pro", "AutoCAD", "Surveying", "Revit"],
    bio: "Civil engineering student researching sustainable concrete
alternatives and eco-friendly structural design.",
    achievement: "State Level Bridge Design Championship - 1st Prize",
    isFollowing: false
  },
  {
    id: 7,
    studentId: "KBT-2024-IN01",
    name: "Tanmay Shinde",
    department: "Instrumentation Engineering",
    deptCode: "instru",
    year: "Third Year (TE)",
    cgpa: 8.56,
    skills: ["PLC Programming", "SCADA", "LabVIEW", "Sensors", "Automation"],
    bio: "Industrial automation enthusiast designing automated quality-control
rigs and industrial telemetry systems.",
    achievement: "Certified Automation Associate by National Council 2024",
    isFollowing: false
  },
  {
    id: 8,
    studentId: "KBT-2023-CS09",
    name: "Ananya Wagh",
    department: "Computer Engineering",
    deptCode: "comp",
    year: "Third Year (TE)",
    cgpa: 9.05,
    skills: ["Flutter", "Dart", "Firebase", "UI/UX Design"],
    bio: "Cross-platform mobile developer dedicated to crafting clean,
intuitive, and accessible user experiences.",
    achievement: "Developed KBTCOE Campus Navigation App with 1500+ active
users",
    isFollowing: false
  },
  {
    id: 9,
    studentId: "KBT-2024-AI08",
    name: "Karan Bhamare",
    department: "Artificial Intelligence & Data Science",
    deptCode: "aids",
```

```
    year: "Second Year (SE)",
    cgpa: 8.90,
    skills: ["TensorFlow", "Pandas", "Scikit-Learn", "SQL", "Python"],
    bio: "Machine learning practitioner passionate about predictive analytics,
financial modeling, and algorithmic trading.",
    achievement: "Kaggle Competitions Expert (Top 5% globally in tabular
models)",
    isFollowing: false
  },
  {
    id: 10,
    studentId: "KBT-2023-IT11",
    name: "Shruti Pawar",
    department: "Information Technology",
    deptCode: "it",
    year: "Final Year (BE)",
    cgpa: 8.72,
    skills: ["Cybersecurity", "Ethical Hacking", "Linux", "Network Security"],
    bio: "Security researcher and CTF player focused on web application
penetration testing and vulnerability disclosure.",
    achievement: "Recognized in Hall of Fame by 3 major tech bug bounty
programs",
    isFollowing: false
  },
  {
    id: 11,
    studentId: "KBT-2024-CS14",
    name: "Vikram More",
    department: "Computer Engineering",
    deptCode: "comp",
    year: "Second Year (SE)",
    cgpa: 8.68,
    skills: ["Go", "Kubernetes", "Microservices", "REST APIs"],
    bio: "Backend systems enthusiast exploring distributed systems, low-latency
networking, and container orchestration.",
    achievement: "Core contributor to popular open-source Go routing framework",
    isFollowing: false
  },
  {
    id: 12,
    studentId: "KBT-2023-ET12",
    name: "Gayatri Sonawane",
    department: "Electronics & Telecommunication",
    deptCode: "entc",
    year: "Third Year (TE)",
    cgpa: 8.81,
    skills: ["DSP", "FPGA", "VHDL", "MATLAB", "Communication Systems"],
    bio: "Wireless communications and digital signal processing enthusiast
researching 5G/6G beamforming algorithms.",
    achievement: "Winner - KBTCOE Annual Hardware Innovation Hackathon 2024",
    isFollowing: false
  }
];
```

```
//
================================================================================
// 2. STATE MANAGEMENT (Demonstrating `let` for mutable state)
//
================================================================================
// We maintain an in-memory mutable copy of the student data for session
updates.
let students = [...initialStudents];

// Current filter criteria
let currentSearchTerm = "";
let currentSelectedBranch = "all";

//
================================================================================
// 3. STATISTICAL CALCULATIONS (Demonstrating reduce, map, filter, Set)
//
================================================================================

/**
 * Calculates distinct branches count using map() and new Set().
 * @param {Array<Object>} studentList - List of student records
 * @returns {number} Number of unique branches
 */
const getDistinctBranchesCount = (studentList = students) => {
  // Arrow function with default parameter
  const branchNames = studentList.map(student => student.department); // map()
demonstration
  const uniqueBranches = new Set(branchNames);
  return uniqueBranches.size;
};

/**
 * Calculates distinct skills count using reduce() and Set.
 * @param {Array<Object>} studentList - List of student records
 * @returns {number} Number of unique technical skills
 */
const getDistinctSkillsCount = (studentList = students) => {
  // reduce() demonstration: aggregates array of skills across all students into
a single Set
  const skillSet = studentList.reduce((accumulatorSet, currentStudent) => {
    currentStudent.skills.forEach(skill => accumulatorSet.add(skill));
    return accumulatorSet;
  }, new Set());

  return skillSet.size;
};

/**
 * Computes average CGPA using reduce() and arithmetic division.
 * Demonstrates operators (+, /), ternary conditional, and toFixed().
 * @param {Array<Object>} studentList - List of student records
 * @returns {string} Formatted average CGPA
 */
```

```
const calculateAverageCgpa = (studentList = students) => {
  if (!studentList || studentList.length === 0) {
    return "0.00";
  }

  const totalCgpa = studentList.reduce((sum, s) => sum + s.cgpa, 0);
  const average = totalCgpa / studentList.length; // Arithmetic division
operator
  return average.toFixed(2);
};

//
============================================================================
// 4. FILTERING LOGIC (Demonstrating filter(), toLowerCase(), includes(), &&,
||)
//
============================================================================

/**
 * Filters the students array based on search term and branch selection.
 * Search criteria searches across:
 * - Student name
 * - Department
 * - Bio
 * - Skills array (via some() and includes())
 * Both filters (search AND branch) work together.
 *
 * @param {string} searchTerm - User search keyword
 * @param {string} branch - Selected department filter ("all" or specific name)
 * @returns {Array<Object>} Filtered list of students
 */
function filterStudents(searchTerm = "", branch = "all") {
  const normalizedQuery = searchTerm.trim().toLowerCase();

  return students.filter(student => {
    // 1. Branch filter condition
    // Comparison operator (===) and logical OR (||)
    const matchesBranch = (branch === "all" || student.department === branch);

    // If query is empty, only check branch match
    if (!normalizedQuery) {
      return matchesBranch;
    }

    // 2. Search query conditions across name, bio, department, and skills
    const matchesName = student.name.toLowerCase().includes(normalizedQuery);
    const matchesDept =
student.department.toLowerCase().includes(normalizedQuery);
    const matchesBio = student.bio.toLowerCase().includes(normalizedQuery);
    const matchesSkills = student.skills.some(skill =>
      skill.toLowerCase().includes(normalizedQuery)
    );

    // Combined search match (logical OR ||)
```

```
    const matchesSearch = matchesName || matchesDept || matchesBio ||
matchesSkills;

    // Must satisfy BOTH branch filter AND search criteria (logical AND &&)
    return matchesBranch && matchesSearch;
  });
}

//
================================================================================
// 5. DOM RENDERING & CARD GENERATION (Safe textContent, createElement)
//
================================================================================

/**
 * Creates a single DOM student card safely without injecting untrusted HTML.
 * Uses createElement, textContent, classList, and appendChild.
 *
 * @param {Object} student - Student record object
 * @returns {HTMLElement} The card article element
 */
function createStudentCardElement(student) {
  // Container card
  const card = document.createElement("article");
  card.className = "student-card";
  card.setAttribute("data-id", String(student.id));

  // --- Top Row: Name, ID, Department Badge ---
  const cardTop = document.createElement("div");
  cardTop.className = "card-top";

  const nameIdWrapper = document.createElement("div");

  const nameEl = document.createElement("h3");
  nameEl.className = "student-name";
  nameEl.textContent = student.name; // Safe assignment

  const idEl = document.createElement("span");
  idEl.className = "student-id";
  idEl.textContent = student.studentId;

  nameIdWrapper.appendChild(nameEl);
  nameIdWrapper.appendChild(idEl);

  const deptBadge = document.createElement("span");
  // Dynamic class assignment based on deptCode
  deptBadge.className = `dept-badge badge-${student.deptCode}`;
  deptBadge.textContent = student.department;

  cardTop.appendChild(nameIdWrapper);
  cardTop.appendChild(deptBadge);

  // --- Student Meta (Year & Branch) ---
  const metaEl = document.createElement("p");
```

```
metaEl.className = "student-meta";
// Template literal demonstration:
metaEl.textContent = `${student.year} • ${student.department}`;

// --- Bio ---
const bioEl = document.createElement("p");
bioEl.className = "student-bio";
bioEl.textContent = student.bio;

// --- Achievement Highlight ---
const achievementEl = document.createElement("div");
achievementEl.className = "achievement-highlight";
achievementEl.textContent = `🏆 ${student.achievement}`;

// --- Skills Section ---
const skillsWrapper = document.createElement("div");
skillsWrapper.className = "skills-wrapper";

const skillsLabel = document.createElement("div");
skillsLabel.className = "skills-label";
skillsLabel.textContent = "Skills & Expertise";
skillsWrapper.appendChild(skillsLabel);

const skillsFlex = document.createElement("div");
skillsFlex.className = "skills-flex";

// Iterate over skills array
student.skills.forEach(skill => {
  const skillPill = document.createElement("span");
  skillPill.className = "skill-pill";
  skillPill.textContent = skill;
  skillsFlex.appendChild(skillPill);
});
skillsWrapper.appendChild(skillsFlex);

// --- Card Bottom: CGPA and Follow Button ---
const cardBottom = document.createElement("div");
cardBottom.className = "card-bottom";

const cgpaPill = document.createElement("span");
cgpaPill.className = "cgpa-pill";
// Template literal with Number formatting:
cgpaPill.textContent = `CGPA: ${student.cgpa.toFixed(2)}`;

// Follow Action Button
const followBtn = document.createElement("button");
followBtn.type = "button";
followBtn.className = "follow-btn";
followBtn.setAttribute("data-id", String(student.id));
followBtn.setAttribute("aria-label", `Follow ${student.name}`);

// Conditional button label and class using ternary operator:
if (student.isFollowing) {
  followBtn.classList.add("following");
```

```
    followBtn.textContent = "✓ Following";
  } else {
    followBtn.textContent = "+ Follow";
  }

  // Follow Button Event Listener
  followBtn.addEventListener("click", () => {
    handleToggleFollow(student.id, followBtn);
  });

  cardBottom.appendChild(cgpaPill);
  cardBottom.appendChild(followBtn);

  // Assemble the card
  card.appendChild(cardTop);
  card.appendChild(metaEl);
  card.appendChild(bioEl);
  card.appendChild(achievementEl);
  card.appendChild(skillsWrapper);
  card.appendChild(cardBottom);

  return card;
}

/**
 * Creates the empty state display when no students match the query.
 * @returns {HTMLElement} Empty state container element
 */
function createEmptyStateElement() {
  const emptyContainer = document.createElement("div");
  emptyContainer.className = "empty-state";

  const iconEl = document.createElement("span");
  iconEl.className = "empty-icon";
  iconEl.textContent = "🔍";

  const titleEl = document.createElement("h3");
  titleEl.className = "empty-title";
  titleEl.textContent = "No Matching Students Found";

  const descEl = document.createElement("p");
  descEl.className = "empty-desc";
  // Template literal for dynamic message:
  descEl.textContent = currentSearchTerm
    ? `No students found matching "${currentSearchTerm}" in the selected
department. Try a different skill, name, or clear your filters.`
    : "No students currently listed in this department. Try selecting another
branch.";

  const resetBtn = document.createElement("button");
  resetBtn.type = "button";
  resetBtn.className = "btn-reset";
  resetBtn.textContent = "Clear All Filters";
  resetBtn.addEventListener("click", handleResetFilters);
```

```
  emptyContainer.appendChild(iconEl);
  emptyContainer.appendChild(titleEl);
  emptyContainer.appendChild(descEl);
  emptyContainer.appendChild(resetBtn);

  return emptyContainer;
}

/**
 * Renders the student cards grid given an array of student records.
 * Handles DOM clearing, appending, and empty state.
 *
 * @param {Array<Object>} studentList - Students to display
 */
function renderStudentCards(studentList) {
  try {
    const studentGrid = document.getElementById("studentGrid");
    if (!studentGrid) {
      console.warn("Target element #studentGrid not found in document.");
      return;
    }

    // Clear existing content
    studentGrid.replaceChildren();

    // Check if empty
    if (!studentList || studentList.length === 0) {
      studentGrid.appendChild(createEmptyStateElement());
      updateResultsSummary(0);
      return;
    }

    // Append each student card
    const fragment = document.createDocumentFragment();
    studentList.forEach(student => {
      const cardEl = createStudentCardElement(student);
      fragment.appendChild(cardEl);
    });

    studentGrid.appendChild(fragment);
    updateResultsSummary(studentList.length);
  } catch (error) {
    console.error("Error occurred while rendering student cards:", error);
  }
}

/**
 * Updates the live results count and filter status indicators.
 * @param {number} count - Matching student count
 */
function updateResultsSummary(count) {
  const countEl = document.getElementById("resultsCount");
  const statusEl = document.getElementById("filterStatus");
```

```
  if (countEl) {
    countEl.textContent = String(count);
  }

  if (statusEl) {
    // Ternary operator demonstration
    const branchLabel = currentSelectedBranch === "all" ? "All Branches" :
currentSelectedBranch;
    const searchLabel = currentSearchTerm ? `for "${currentSearchTerm}"` : "";
    statusEl.textContent = `Filtered by: ${branchLabel} ${searchLabel}`;
  }

  // Update Matching Count in Dashboard Stat Card
  const matchingStatEl = document.getElementById("statMatchingStudents");
  if (matchingStatEl) {
    matchingStatEl.textContent = String(count);
  }
}

/**
 * Updates top dashboard statistics cards.
 */
function updateDashboardStats() {
  try {
    const totalEl = document.getElementById("statTotalStudents");
    const matchingEl = document.getElementById("statMatchingStudents");
    const branchesEl = document.getElementById("statBranches");
    const skillsEl = document.getElementById("statSkills");

    if (totalEl) {
      totalEl.textContent = String(students.length);
    }
    if (matchingEl) {
      matchingEl.textContent = String(students.length);
    }
    if (branchesEl) {
      branchesEl.textContent = String(getDistinctBranchesCount(students));
    }
    if (skillsEl) {
      skillsEl.textContent = String(getDistinctSkillsCount(students));
    }
  } catch (error) {
    console.error("Error updating dashboard statistics:", error);
  }
}

//
============================================================================
// 6. EVENT HANDLERS (Demonstrating event loop, find, state mutation)
//
============================================================================

/**
```

```
 * Toggles the follow status for a specific student.
 * Demonstrates find() array method and conditional state toggling.
 *
 * @param {number} studentId - Unique ID of the student
 * @param {HTMLButtonElement} buttonElement - Button element that triggered the
event
 */
function handleToggleFollow(studentId, buttonElement) {
  try {
    // Array method find() demonstration
    const student = students.find(s => s.id === studentId);

    if (!student) {
      console.error(`Student with id ${studentId} not found.`);
      return;
    }

    // Toggle boolean value using NOT operator (!)
    student.isFollowing = !student.isFollowing;

    // Update button UI reflecting the new state
    if (student.isFollowing) {
      buttonElement.classList.add("following");
      buttonElement.textContent = "✓ Following";
    } else {
      buttonElement.classList.remove("following");
      buttonElement.textContent = "+ Follow";
    }

    // Visual feedback in console demonstrating object property inspection
    console.log(`[Follow State Changed] ID: ${student.id}, Name:
${student.name}, Following: ${student.isFollowing}`);
  } catch (error) {
    console.error("Error toggling follow state:", error);
  }
}

/**
 * Handles search input changes with immediate filtering.
 * @param {Event} event - Input event
 */
function handleSearchInput(event) {
  currentSearchTerm = event.target.value;
  applyFiltersAndRender();
}

/**
 * Handles branch dropdown changes.
 * @param {Event} event - Change event
 */
function handleBranchChange(event) {
  currentSelectedBranch = event.target.value;
  applyFiltersAndRender();
}
```

```
/**
 * Resets search input and branch dropdown back to initial default state.
 */
function handleResetFilters() {
  currentSearchTerm = "";
  currentSelectedBranch = "all";

  const searchInput = document.getElementById("searchInput");
  const branchFilter = document.getElementById("branchFilter");

  if (searchInput) {
    searchInput.value = "";
    searchInput.focus();
  }

  if (branchFilter) {
    branchFilter.value = "all";
  }

  applyFiltersAndRender();
}

/**
 * Centralized filter application function.
 */
function applyFiltersAndRender() {
  const filtered = filterStudents(currentSearchTerm, currentSelectedBranch);
  renderStudentCards(filtered);
}

//
============================================================================
// 7. INITIALIZATION & SETUP
//
============================================================================

/**
 * Populates the branch dropdown dynamically using distinct departments.
 * Demonstrates map(), Set, forEach(), and createElement.
 */
function populateBranchDropdown() {
  try {
    const branchSelect = document.getElementById("branchFilter");
    if (!branchSelect) return;

    // Get unique departments using map and Set
    const uniqueDepartments = Array.from(new Set(students.map(s =>
s.department)));
    uniqueDepartments.sort();

    // Preserve the first "All Branches" option, then append unique departments
    uniqueDepartments.forEach(dept => {
      const option = document.createElement("option");
```

```
      option.value = dept;
      option.textContent = dept;
      branchSelect.appendChild(option);
    });
  } catch (error) {
    console.error("Error populating branch dropdown:", error);
  }
}

/**
 * Initializes the application on DOMContentLoaded.
 * Attaches event listeners and triggers initial rendering.
 */
function initializeApp() {
  try {
    console.log("Initializing KBTCOE Connect Student Directory (Assignment
06)...");

    // 1. Populate branch dropdown with unique values
    populateBranchDropdown();

    // 2. Render initial dashboard statistics
    updateDashboardStats();

    // 3. Render full initial list of students
    renderStudentCards(students);

    // 4. Attach event listeners
    const searchInput = document.getElementById("searchInput");
    if (searchInput) {
      searchInput.addEventListener("input", handleSearchInput);
    }

    const branchFilter = document.getElementById("branchFilter");
    if (branchFilter) {
      branchFilter.addEventListener("change", handleBranchChange);
    }

    const resetBtn = document.getElementById("resetBtn");
    if (resetBtn) {
      resetBtn.addEventListener("click", handleResetFilters);
    }

    console.log("Student Directory initialized successfully with",
students.length, "profiles.");
  } catch (error) {
    console.error("Fatal initialization error:", error);
  }
}

// Ensure the DOM is fully loaded before executing initialization (Browser
environment check)
if (typeof document !== "undefined") {
  if (document.readyState === "loading") {
```

```
    document.addEventListener("DOMContentLoaded", initializeApp);
  } else {
    // DOM is already ready
    initializeApp();
  }
}

// Export for Node.js unit testing environments (if running in test runner)
if (typeof module !== "undefined" && module.exports) {
  module.exports = {
    initialStudents,
    students,
    getDistinctBranchesCount,
    getDistinctSkillsCount,
    calculateAverageCgpa,
    filterStudents,
    handleToggleFollow
  };
}
```